CodeIgniter 4: A Comprehensive Guide

1. Introduction to CodeIgniter 4

CodeIgniter 4 is a modern, powerful, and lightweight PHP full-stack web framework designed for developers who need a simple and elegant toolkit to create full-featured web applications. As a significant rewrite from its predecessors, CodeIgniter 4 focuses on performance, security, and developer productivity. It provides a rich set of libraries for common tasks, along with a logical structure to keep your code organized and maintainable.

Key Features:

- **High Performance**: Optimized for speed, CodeIgniter 4 delivers exceptional performance, making it suitable for demanding applications.
- **Small Footprint**: The framework is lean, requiring minimal resources, which contributes to faster execution and lower hosting costs.
- **Security**: Built-in security features such as CSRF protection, XSS filtering, and Content Security Policy help developers create robust and secure applications.
- **Clear Documentation**: Comprehensive and well-organized documentation makes it easy for developers to learn and use the framework.
- **MVC Architecture**: Adheres to the Model-View-Controller architectural pattern, promoting separation of concerns and modular development.
- **Command Line Interface (CLI)**: The `Spark` tool provides a powerful command-line interface for various development tasks, including migrations, code generation, and server management.

2. Application Structure

Understanding the directory structure of a CodeIgniter 4 application is crucial for effective development. The framework provides a logical and organized layout that

separates application code from system files and public assets. [1]

Default Directories:

- **`app/`** : This directory contains all your application-specific code. It's where you'll spend most of your development time. It includes subdirectories for configurations, controllers, models, views, and other custom classes.
  - `Config/` : Stores configuration files for the application.
  - `Controllers/` : Houses the controllers that handle incoming requests and orchestrate the application flow.
  - `Database/` : Contains database migrations and seed files.
  - `Models/` : Defines the models that interact with the database and enforce business rules.
  - `Views/` : Holds the view files responsible for rendering the user interface.
- **`public/`** : This is the web root of your application. It contains the `index.php` file, which is the single entry point for all requests, along with public assets like CSS, JavaScript, and images. Web servers should be configured to point to this directory to prevent direct access to sensitive application files.
- **`writable/`** : This directory is used for storing files that need to be written to during the application's lifecycle, such as cache files, log files, and user uploads. Keeping this directory separate enhances security by making other primary directories non-writable.
- **`tests/`** : Contains your application's test files. This directory is typically not deployed to production servers.
- **`system/`** : This directory holds the core framework files. It is strongly advised **not** to modify any files within this directory. Instead, extend existing classes or create new ones to customize functionality.
- **`vendor/`** : (If installed via Composer) Contains third-party libraries and dependencies managed by Composer.

3. Model-View-Controller (MVC) Architecture

CodeIgniter 4 follows the Model-View-Controller (MVC) architectural pattern, which separates an application into three main logical components: the Model, the View, and the Controller. This separation of concerns makes applications more modular, easier to understand, and simpler to maintain. [2]

Components of MVC:

- **Model**: Models are responsible for managing the application's data and enforcing business rules. They interact with the database to retrieve, store, and update information. In CodeIgniter, models are typically located in `app/Models`.

  Example Model (`app/Models/UserModel.php`):

```php
<?php namespace App\Models;


use CodeIgniter\Model;


class UserModel extends Model
{
    protected $table      = 'users';
    protected $primaryKey = 'id';


    protected $useAutoIncrement = true;


    protected $returnType     = 'array';
    protected $useSoftDeletes = true;


    protected $allowedFields = ['name', 'email'];


    protected $useTimestamps = false;
    protected $createdField  = 'created_at';
    protected $updatedField  = 'updated_at';
    protected $deletedField  = 'deleted_at';


    protected $validationRules    = [];
    protected $validationMessages = [];
    protected $skipValidation     = false;
}
```

- **View**: Views are responsible for presenting data to the user. They primarily consist of HTML, with minimal PHP code used for displaying variables or looping through data. Views should not contain complex business logic. In CodeIgniter, views are typically stored in `app/Views`.

  Example View (`app/Views/users/overview.php`):

```
<!DOCTYPE html>
<html>
<head>
    <title>User Overview</title>
</head>
<body>
    <h1>Welcome, <?= esc($name) ?>!</h1>
    <p>Your email is: <?= esc($email) ?></p>
</body>
</html>
```

- **Controller**: Controllers act as intermediaries between models and views. They receive user input, process it (often by interacting with models), and then pass the data to the appropriate view for display. Controllers also handle HTTP requests, routing, and security. In CodeIgniter, controllers are located in `app/Controllers`.

  Example Controller (`app/Controllers/Home.php`):

```
<?php namespace App\Controllers;


class Home extends BaseController
{
    public function index()
    {
        $data = [
            'name' => 'John Doe',
            'email' => 'john.doe@example.com'
        ];
        return view('users/overview', $data);
    }
}
```

4. Routing in CodeIgniter 4

Routing is the process of mapping incoming HTTP requests (URIs) to specific controller methods. CodeIgniter 4 provides a flexible and powerful routing system that allows

you to define custom routes, use placeholders, and handle different HTTP verbs. [3]

Routes are defined in the `app/Config/Routes.php` file. The `$routes` object, an instance of `RouteCollection`, is used to specify routing rules.

Basic Routing Example:

```php
<?php

use CodeIgniter\Router\RouteCollection;

/**
 * @var RouteCollection $routes
 */
$routes->get('/', 'Home::index');
$routes->get('/about', 'Home::about');
$routes->get('/users/(:num)', 'Users::show/$1');
$routes->post('/users', 'Users::create');
```

In this example:

- `$routes->get('/', 'Home::index');` maps a GET request to the root URL (`/`) to the `index` method of the `Home` controller.
- `$routes->get('/about', 'Home::about');` maps a GET request to `/about` to the `about` method of the `Home` controller.
- `$routes->get('/users/(:num)', 'Users::show/$1');` maps a GET request to `/users/` followed by a number (e.g., `/users/123`) to the `show` method of the `Users` controller, passing the number as an argument. `(:num)` is a placeholder that matches any positive integer.
- `$routes->post('/users', 'Users::create');` maps a POST request to `/users` to the `create` method of the `Users` controller.

Placeholders:

CodeIgniter 4 provides several placeholders for defining dynamic segments in your routes:

Placeholder	Description
`(:any)`	Matches all characters from that point to the end of the URI.
`(:segment)`	Matches any character except a forward slash (`/`), restricting to a single segment.
`(:num)`	Matches any positive integer.
`(:alpha)`	Matches any string of alphabetic characters.
`(:alphanum)`	Matches any string of alphanumeric characters.
`(:hash)`	Same as `(:segment)`, often used for hashed IDs.

HTTP Verb Routes:

You can define routes that respond to specific HTTP methods (GET, POST, PUT, DELETE, etc.) using methods like `get()`, `post()`, `put()`, `delete()`, or `match()` for multiple verbs.

```php
<?php

// Matches only POST requests to /products
$routes->post('products', 'Product::feature');

// Matches only PUT requests to /products/1
$routes->put('products/1', 'Product::feature');

// Matches GET or PUT requests to /products
$routes->match(['get', 'put'], 'products', 'Product::feature');
```

5. Database Interaction

CodeIgniter 4 offers a robust and easy-to-use database abstraction layer that supports various database systems. It provides both a traditional query interface and a powerful Query Builder class, simplifying database operations. [4]

Database Configuration:

Database connection settings are typically configured in `app/Config/Database.php` or within your `.env` file for environment-specific settings.

Example `.env` configuration:

```
DATABASE.default.hostname = localhost
DATABASE.default.database = ci4_database
DATABASE.default.username = root
DATABASE.default.password =
DATABASE.default.DBDriver = MySQLi
```

Connecting to the Database:

You can connect to the database manually or configure it for automatic connection.

```php
<?php

$db = \Config\Database::connect();
```

Query Builder Class:

The Query Builder provides a convenient way to build and execute database queries without writing raw SQL. It supports common operations like `SELECT`, `INSERT`, `UPDATE`, and `DELETE`.

Example using Query Builder:

```php
<?php namespace App\Controllers;

use App\Models\UserModel;
use CodeIgniter\Controller;

class Users extends Controller
{
    public function index()
    {
        $userModel = new UserModel();
        $users = $userModel->findAll(); // Select all users

        // Or using Query Builder directly:
        // $db = \Config\Database::connect();
        // $users = $db->table('users')->get()->getResultArray();

        return view('users/list', ['users' => $users]);
    }

    public function create()
    {
        $userModel = new UserModel();
        $data = [
            'name'  => $this->request->getPost('name'),
            'email' => $this->request->getPost('email')
        ];

        $userModel->insert($data);

        return redirect()->to('/users');
    }
}
```

Standard Queries:

For more complex queries or when you prefer writing raw SQL, you can use the standard query method.

```php
<?php

$db = \Config\Database::connect();

$query = $db->query('SELECT name, email FROM users WHERE status = ?',
['active']);
$results = $query->getResultArray();

foreach ($results as $row) {
    echo $row['name'] . ' - ' . $row['email'] . '<br>';
}
```

6. Code Examples: Building a Simple Blog

Let's walk through a basic example of creating a simple blog application using CodeIgniter 4. This will demonstrate the interaction between models, views, and controllers, along with routing and database operations.

Step 1: Database Setup

First, create a database named `ci4_blog` and a `posts` table.

```php
CREATE TABLE `posts` (
    `id` INT(11) NOT NULL AUTO_INCREMENT,
    `title` VARCHAR(255) NOT NULL,
    `content` TEXT NOT NULL,
    `created_at` DATETIME DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

Update your `.env` file with the database credentials.

Step 2: Create a Post Model

Create `app/Models/PostModel.php`:

```php
<?php namespace App\Models;

use CodeIgniter\Model;

class PostModel extends Model
{
    protected $table      = 'posts';
    protected $primaryKey = 'id';

    protected $useAutoIncrement = true;

    protected $returnType     = 'array';
    protected $useSoftDeletes = false;

    protected $allowedFields = ['title', 'content'];

    protected $useTimestamps = true;
    protected $createdField  = 'created_at';
    protected $updatedField  = 'updated_at';
    protected $deletedField  = 'deleted_at';

    protected $validationRules    = [];
    protected $validationMessages = [];
    protected $skipValidation     = false;
}
```

Step 3: Create a Posts Controller

Create `app/Controllers/Posts.php`:

```php
<?php namespace App\Controllers;

use App\Models\PostModel;
use CodeIgniter\Controller;

class Posts extends Controller
{
    public function index()
    {
        $postModel = new PostModel();
        $data['posts'] = $postModel->findAll();
        return view('posts/index', $data);
    }

    public function view($id = null)
    {
        $postModel = new PostModel();
        $data['post'] = $postModel->find($id);

        if (empty($data['post']))
        {
            throw new \CodeIgniter\Exceptions\PageNotFoundException('Cannot
find the post item: ' . $id);
        }

        return view('posts/view', $data);
    }

    public function create()
    {
        helper('form');

        if ($this->request->getMethod() === 'post' && $this->validate([
            'title' => 'required|min_length[3]|max_length[255]',
            'content'  => 'required'
        ]))
        {
            $postModel = new PostModel();
            $postModel->save([
                'title' => $this->request->getPost('title'),
                'content'  => $this->request->getPost('content'),
            ]);

            return view('posts/success');
        }
```

```
        return view('posts/create');
    }
}
```

Step 4: Create Views

Create `app/Views/posts/index.php`:

```
<!DOCTYPE html>
<html>
<head>
    <title>Blog Posts</title>
</head>
<body>
    <h1>Blog Posts</h1>

    <?php if (! empty($posts) && is_array($posts)): ?>

        <?php foreach ($posts as $post): ?>

            <h3><?= esc($post['title']) ?></h3>
            <div class="main">
                <?= esc(word_limiter($post['content'], 20)) ?>
            </div>
            <p><a href="/posts/view/<?= esc($post['id'], 'url') ?>">View
article</a></p>

        <?php endforeach; ?>

    <?php else: ?>

        <h3>No Posts</h3>

        <p>Unable to find any posts for you.</p>

    <?php endif ?>
</body>
</html>
```

Create `app/Views/posts/view.php`:

```
<!DOCTYPE html>
<html>
<head>
    <title><?= esc($post['title']) ?></title>
</head>
<body>
    <h1><?= esc($post['title']) ?></h1>
    <p><?= esc($post['content']) ?></p>
</body>
</html>
```

Create `app/Views/posts/create.php`:

```
<!DOCTYPE html>
<html>
<head>
    <title>Create a new post</title>
</head>
<body>
    <h1>Create a new post</h1>

    <?= validation_list_errors() ?>

    <form action="/posts/create" method="post">
        <?= csrf_field() ?>

        <label for="title">Title</label>
        <input type="input" name="title" value="<?= set_value('title') ?>">
<br>

        <label for="content">Content</label>
        <textarea name="content"><?= set_value('content') ?></textarea><br>

        <input type="submit" name="submit" value="Create post">
    </form>
</body>
</html>
```

Create `app/Views/posts/success.php`:

```
<!DOCTYPE html>
<html>
<head>
    <title>Post Created</title>
</head>
<body>
    <h1>Post Created Successfully!</h1>
    <p><a href="/posts">View all posts</a></p>
</body>
</html>
```

Step 5: Define Routes

Update `app/Config/Routes.php` to include routes for the blog posts:

```php
<?php

use CodeIgniter\Router\RouteCollection;

/**
 * @var RouteCollection $routes
 */
$routes->get('/', 'Home::index');

$routes->get('posts', 'Posts::index');
$routes->get('posts/create', 'Posts::create');
$routes->post('posts/create', 'Posts::create');
$routes->get('posts/view/(:segment)', 'Posts::view/$1');
```

7. Conclusion

CodeIgniter 4 provides a robust, high-performance, and developer-friendly framework for building modern PHP web applications. Its adherence to the MVC pattern, flexible routing, and powerful database abstraction make it an excellent choice for projects of all sizes. By understanding its core concepts and leveraging its features, developers can efficiently create secure and scalable applications.

References

[1] CodeIgniter4 Overview. *CodeIgniter 4.7.0 documentation*. Available at: https://codeigniter4.github.io/userguide/concepts/index.html [2] Models, Views, and Controllers. *CodeIgniter 4.7.0 documentation*. Available at: https://codeigniter4.github.io/userguide/concepts/mvc.html [3] URI Routing. *CodeIgniter 4.7.0 documentation*. Available at: https://codeigniter4.github.io/userguide/incoming/routing.html [4] Working with Databases. *CodeIgniter 4.7.0 documentation*. Available at: https://codeigniter4.github.io/userguide/database/index.html